Project - High Level Design

On

Real Estate Support Triage Agent

Course Name:

Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	CHERRY MEHTA	EN22CS301292
2	BHUMIKA CHOYAL	EN22CS301276
3	AYUSH JAIN	EN22CS301247
4	AYUSH PATIDAR	EN22CS301253
5	DIYA GOYAL	EN22CS301351

Group Name: 03D3

Project Number: AAI-27

Industry Mentor Name: Mr. Ashwin Krishnan Ck

University Mentor Name: Prof. Maya Yadav Baniya

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. State and Session Management

5. Interfaces

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the document

The purpose of this document is to describe the design, architecture, and functionality of the Real Estate Support Triage Agent, an AI-based system developed to automate the handling of customer communications in the real estate domain.

The system processes customer messages such as inquiries, complaints, and service requests. It uses Artificial Intelligence and Natural Language Processing techniques to classify messages, determine urgency, extract key information, and generate appropriate responses.

This document covers the system architecture, application workflow, process flow, data handling, and key components involved in the system. It serves as a reference for understanding how the system operates and how different modules interact.

1.2 Intended Audience

This document is intended for:

- Software Developers and Engineers responsible for building and maintaining the system
- AI/ML Engineers working on text classification, NER, and LLM integration
- Students and Researchers studying AI-based automation systems
- Real Estate Support Teams and Organizations interested in improving customer communication workflows

The document provides both conceptual and technical understanding of the system.

1.3 System Overview

The Real Estate Support Triage Agent is an intelligent system designed to automate customer support in the real estate domain.

The system receives customer messages from different sources such as chat, email, or web forms. It processes these messages using AI models to:

- Identify the intent (Inquiry, Complaint, Request)
- Determine the urgency level (High, Medium, Low)

- Extract important details such as property ID, date, and location
- Generate a response using a Large Language Model (LLM)

Technology Stack and System Architecture

The proposed system is built using a combination of modern technologies to ensure efficient performance, scalability, and user interaction.

Backend Development: The backend of the application is implemented using Python, which provides robust support for handling server-side logic, data processing, and integration with machine learning models.

Large Language Models (LLMs): Advanced LLMs such as OpenAI and Gemini are utilized for response generation. These models enable the system to produce context-aware, accurate, and meaningful outputs based on user inputs.

Natural Language Processing (NLP): NLP models are incorporated for tasks such as text classification and entity extraction. These functionalities help in understanding user queries, identifying relevant information, and improving the overall interaction quality.

User Interface: The front-end interface is developed using Streamlit or other web-based UI frameworks. This ensures a user-friendly and interactive environment for seamless communication between the user and the system.

Database Management: Data storage is handled using databases such as MongoDB. These databases support efficient data retrieval, storage, and management of structured and unstructured data.

REST APIs: RESTful APIs are used to facilitate communication between different components of the system. They ensure smooth data exchange and integration across the backend, frontend, and AI modules.

2. System Design

2.1 Application Design

The Real Estate Support Triage Agent follows a layered architecture similar to your reference format.

2.1.1. User Interface (Frontend Layer)

This layer allows users or support teams to interact with the system.

Responsibilities:

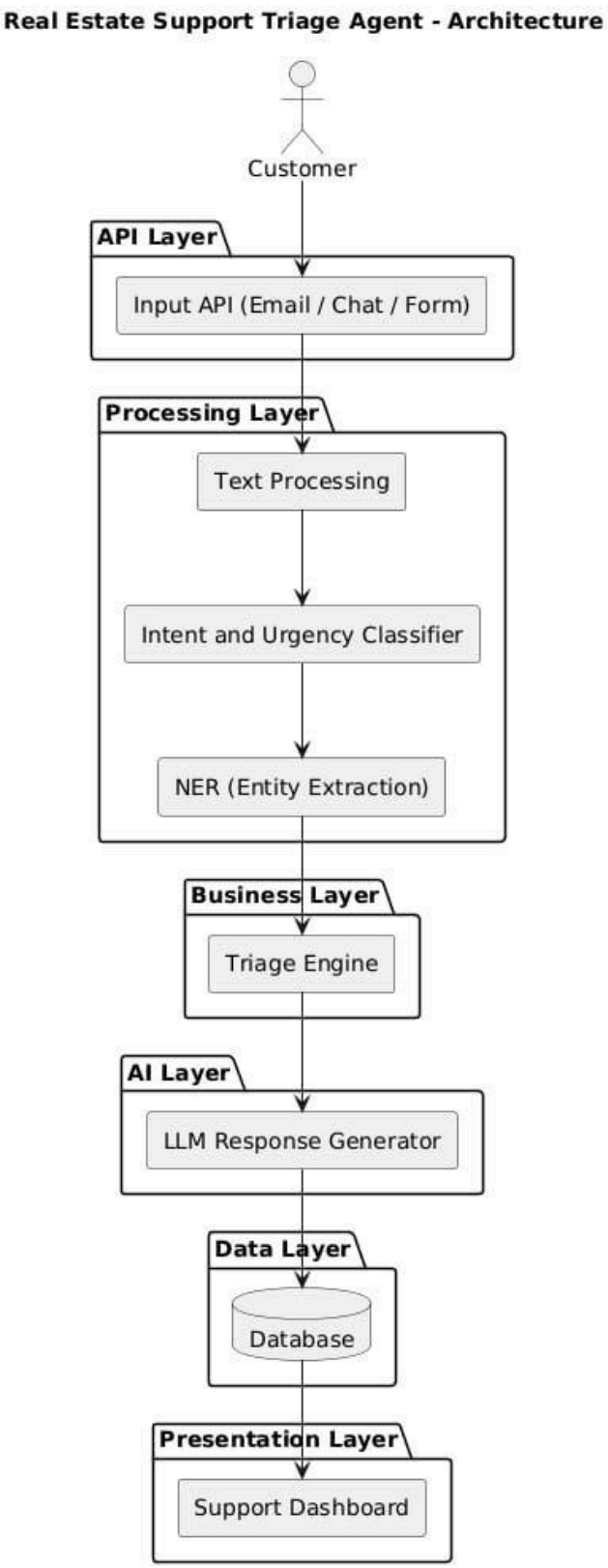
- Accept customer messages
- Display classified results (intent & urgency)
- Show extracted details
- Display generated responses

2.1.2. Backend Layer (Application Logic)

This is the core of the system that processes all operations.

Main Modules:

- Main Controller: Controls overall workflow
- Message Processor: Cleans and validates input
- Classifier Module: Identifies intent and urgency
- NER Module: Extracts important details
- Triage Engine: Assigns priority
- Response Generator: Generates replies using AI



Architecture Diagram

2 1.3. AI Layer

This layer handles intelligent processing using AI models.

Functions:

- Understand message meaning
- Classify intent and urgency
- Extract entities
- Generate responses
- The system uses LLM APIs to produce accurate and context-aware replies.

2.1.4. Data Layer

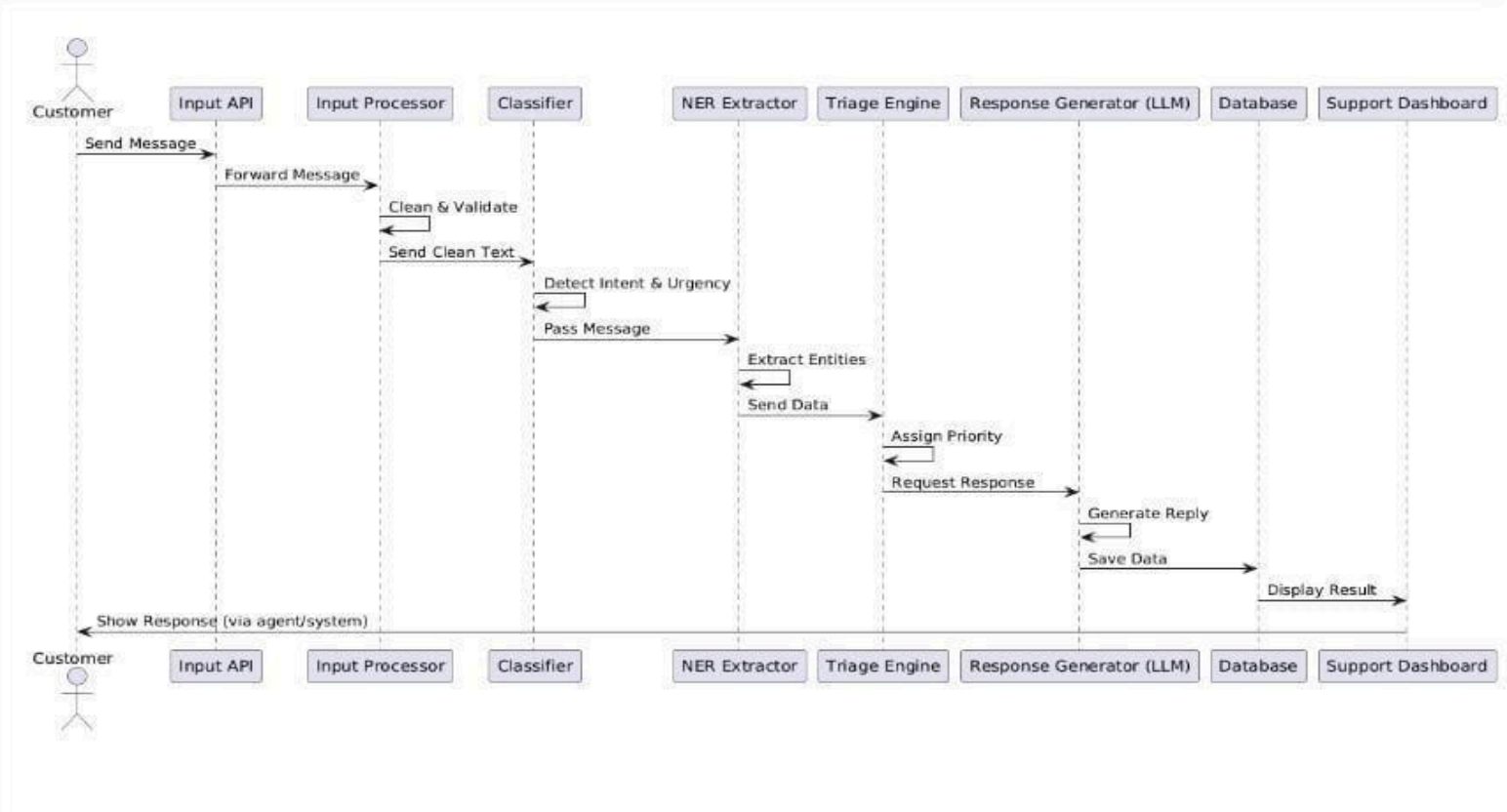
This layer manages storage and retrieval of data.

Components:

- Customer messages
- Extracted entities
- Generated responses
- Property-related data

2.2 Process Flow

1. Customer sends a message
2. System receives the message
3. Message is cleaned and validated
4. Intent and urgency are identified
5. Important details are extracted
6. Message is prioritized
7. AI generates a response
8. Data is stored in database
9. Results are shown on dashboard



Sequence Diagram

2.3 Information Flow

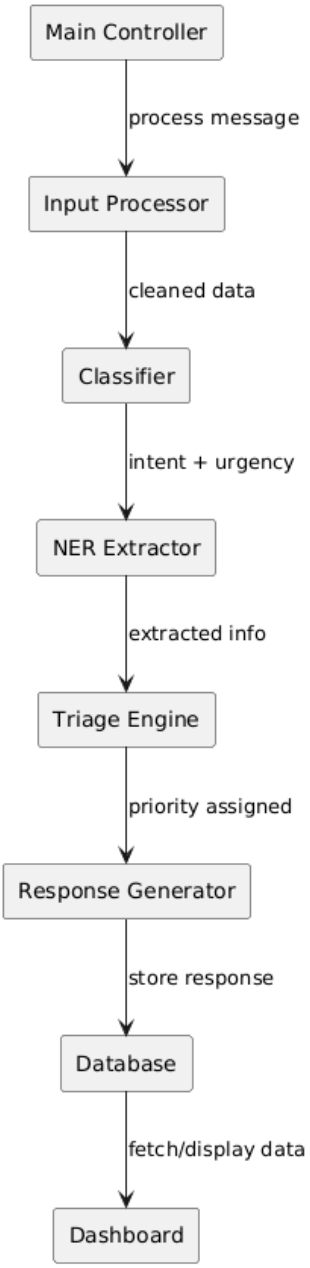
- User Input: Customer sends message
- Processing: System cleans and analyzes message
- Classification: Intent and urgency identified
- Extraction: Key details extracted
- AI Processing: Response generated
- Output: Response shown to support team/user

2.4 Components Design

The system consists of the following components:

- Main Controller: Manages system flow
- Input Processor: Cleans and validates messages
- Classifier: Identifies intent and urgency
- NER Extractor: Extracts key information
- Triage Engine: Assigns priority
- Response Generator: Generates replies

- Database: Stores all data
- Dashboard: Displays result



Component Diagram

2.5 Key Design Considerations

- Scalability: Can handle large number of messages
- Accuracy: AI ensures correct classification and responses
- User Experience: Simple and interactive interface
- Modularity: Easy to update individual components
- Security: API keys and data handled securely
- Performance: Fast processing using optimized workflows

2.6 API Catalogue

API Function Name	Module
receive_message()	Input API
clean_text()	Input Processor
classify_intent()	Classifier
detect_urgency()	Classifier
extract_entities()	NER Module
assign_priority()	Triage Engine
generate_response()	Response Generator
save_date()	Database
fetch_messages()	Dashboard

3. Data Design

3.1 Data Model

The system uses a structured data model to manage customer messages and related information.

Main Data Entities

- Customer: Stores user details
- Message: Stores message content and metadata
- Extracted Data: Contains property ID, date, location, etc.
- Response: Stores generated replies
- Property: Stores property-related information

3.2 Data Access Mechanism

The system follows a simple data access approach:

- Messages are received and processed in real-time
- Data is stored in the database
- Required information is retrieved when needed
- AI models use processed data to generate responses

3.3 Data Retention Policies

- Customer messages are stored temporarily for processing
- Important data such as responses and logs may be stored for analysis
- No sensitive personal data is permanently stored
- Data can be deleted or cleaned periodically

3.4 Data Migration

Data migration involves storing and updating system data:

- Receive customer messages
- Process and clean data
- Extract important details
- Store in database

This process ensures all data is properly structured and available.

4. Session and State Management

4.1. Session Management

- Each user interaction is treated as a session
- Session starts when user sends a message
- No long-term session storage is required

4.2. State Management

- Current message and results are handled temporarily
- System processes each request independently
- No complex state storage is used

4.3. Frontend–Backend Interaction

The frontend communicates with the backend through API calls, sending user responses and receiving evaluation results. Each request is stateless from the backend perspective, meaning the server does not retain session data between requests. The frontend is responsible for maintaining continuity of the user experience by storing and updating session-related information.

5. Interfaces

The system includes the following interfaces:

User Interface

- Used by customers to send messages
- Used by support team to view results

API Interface

- Connects frontend and backend
- Sends and receives data

AI Interface

- Connects system with LLM APIs
- Used for response generation

6. Caching

The system uses basic caching to improve performance:

- Frequently used data can be stored temporarily
- Reduces repeated processing
- Improves response time

Example:

- Previously processed messages
- AI responses (optional caching)

7. Non-Functional Requirements

7.1. Security Aspects

- API keys are stored securely using environment variables
- No hardcoding of sensitive data
- User data is handled safely
- System prevents unauthorized access

7.2. Performance Aspects

- System provides fast response to customer queries
- Efficient processing reduces delays
- AI responses are generated quickly
- System can handle multiple requests

8. References

[1] OpenAI, "OpenAI API Documentation." [Online]. Available: https://platform.openai.com/docs. [Accessed: Mar. 2026].

[2] Hugging Face, "Transformers Documentation." [Online]. Available: https://huggingface.co/docs/transformers. [Accessed: Mar. 2026].

[3] spaCy, "Industrial-Strength Natural Language Processing in Python." [Online]. Available: https://spacy.io/. [Accessed: Mar. 2026].

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805.

[5] Python Software Foundation, “Python Documentation.” [Online]. Available: https://docs.python.org/3/. [Accessed: Mar. 2026].